STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: TEST_Q003_ARITH
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Software Testing Debugging Concepts using AI	STATUS: In Progress

Practice Worksheet – Q003 – Arithmetic Exception Handling

Test and triage runtime arithmetic exceptions in financial division calculators, assert zero-division boundaries with JUnit 5, and design robust exception handling with AI red-teaming.

KEY LEARNING OBJECTIVES:

- Assert runtime ArithmeticException occurrences using JUnit 5 assertThrows().
- Distinguish integer division exceptions from IEEE-754 floating-point Infinity results.
- Implement graceful service-level try-catch-finally fallbacks with audit logging.
- Formulate AI prompts to inspect codebases for unhandled divide-by-zero vulnerabilities.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Examine mathematical calculation methods for potential zero-denominator risks.
Step 2: Formulate unit test assertions verifying exception types and error messages.
Step 3: Design service refactorings that replace unhandled crashes with defensive validation.
Step 4: Document test results and exception propagation paths in the worksheet.

EXERCISE 3.1: INTEGER DIVISION VS FLOATING-POINT ZERO DIVISION ANOMALY

Problem Statement: In Java, `10 / 0` throws `ArithmeticException: / by zero`, but `10.0 / 0.0` returns `Double.POSITIVE_INFINITY` without throwing an exception. Write an analytical comparison detailing why this discrepancy exists and how a financial rate calculation method must defensively guard both integer and double divisions.

Reference Baseline / Scenario / Code:

```
int a = 10 / 0; // Throws ArithmeticException
double b = 10.0 / 0.0; // Evaluates to Infinity
```

STUDENT SOLUTION & CODE WORKSPACE:
[1] IEEE-754 vs Integer Division Analysis: _____
[2] Defensive Guard Clauses: _____
[3] Financial Rounding & Precision Risks: _____

EXERCISE 3.2: JUNIT 5 ASSERTTHROWS EXCEPTION VERIFICATION

Problem Statement: Write a JUnit 5 unit test verifying that `CalculatorService.divide(int numerator, int denominator)` throws `ArithmeticException` when denominator is 0. Assert both the exception class and that `getMessage()` contains '/ by zero'.

Reference Baseline / Scenario / Code:

```
ArithmeticException ex = assertThrows(ArithmeticException.class, () -> calculator.divide(10, 0));
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] assertThrows Lambda Setup: _____

[2] Exception Message Verification: _____

[3] Test Method Code Block: _____

EXERCISE 3.3: SERVICE-LEVEL FALLBACK & CUSTOM DOMAIN EXCEPTION HANDLING

Problem Statement: Refactor a billing calculation method `BigDecimal calculateUnitCost(BigDecimal totalExpense, int unitsProduced)`. When `unitsProduced == 0`, the service must catch the condition before division and throw a custom `ZeroProductionBatchException` with HTTP 400 Bad Request mapping.

Reference Baseline / Scenario / Code:

```
if (unitsProduced == 0) throw new ZeroProductionBatchException("Cannot compute unit cost for 0 items");
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Defensive Precondition Check: _____

[2] Custom Domain Exception Structure: _____

[3] Global Exception Handler Mapping: _____

EXERCISE 3.4: AI PROMPT FOR STATIC VULNERABILITY SCANNING OF ARITHMETIC BUGS

Problem Statement: Construct a prompt for an AI security auditor to scan a Spring Boot codebase for unhandled division operations in billing, inventory consumption, and tax calculation services. The prompt must instruct the AI to flag raw `/` operators lacking prior zero checks.

Reference Baseline / Scenario / Code:

```
Act as a Senior Static Code Security Auditor. Scan the following financial service for division by zero...
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] AI Security Auditor Persona: _____

[2] Code Inspection Directives: _____

[3] Remediation & Patch Output Format: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Exception Type Rigor:** assertThrows explicitly verifies ArithmeticException class.
- [ ] **Message Assertion:** Exception message content verified against expected string.
- [ ] **Floating-Point Guard:** Accounts for Double.isInfinite() and Double.isNaN() checks.
- [ ] **Service Gracefulness:** Prevents 500 Internal Server Errors via custom domain exceptions.
- [ ] **Static Scan Prompt:** AI prompt effectively identifies unguarded division operators.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is returning `Double.POSITIVE_INFINITY` in commercial billing systems more hazardous than throwing an immediate exception?*
- *How does pre-checking inputs (e.g. `if (b == 0)`) compare to catching runtime exceptions in high-throughput systems?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) **[ ] NEEDS REVISION (Resubmit Corrected Work)** **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases